

Programação Competitiva

Aula 4 - Dois Ponteiros e Sliding Window

Adrielly Medeiros

Universidade de Brasília

20 de junho de 2026



Objetivo da aula

Hoje vamos estudar duas técnicas que aparecem muito em problemas de vetores e strings:

- **Two Pointers:** dois índices andando pelo vetor.
- **Sliding Window:** uma janela $[l, r]$ que cresce e encolhe.
- **Pergunta:** quando vocês veem dois for aninhados, qual complexidade normalmente aparece?
- ★ **Dica:** Muitas vezes o objetivo dessas técnicas é transformar uma ideia $O(n^2)$ em $O(n)$ ou $O(n \log n)$.

O que é um “ponteiro” nessa aula?

Aqui, **ponteiro não é ponteiro de memória do C/C++**.

Nesta aula, um ponteiro é só uma variável inteira que guarda uma posição:

$$l = 0, \quad r = n - 1$$

$$v = [1, 3, 4, 6, 8, 12]$$

l

r

► **Pergunta:** No `for(int i=0; i<n; i++)`, o `i` também pode ser visto como um ponteiro?

Ideia central

Two pointers significa:

- manter dois índices importantes;
- mover esses índices com alguma regra;
- evitar testar combinações repetidas ou impossíveis.

Existem dois formatos muito comuns:

- **Pontas opostas:** l começa no início e r no final.
- **Mesma direção:** l e r andam para a direita. Esse caso vira **Sliding Window**.

Quando suspeitar de Two Pointers?

Use essa lista mental:

- O problema fala de **par**, **tripla**, **intervalo**, **subarray** ou **substring**?
- O vetor está **ordenado** ou pode ser ordenado?
- Você está testando todos os pares com dois `for`?
- A condição permite decidir: “movo a esquerda” ou “movo a direita”?
- A resposta depende de um intervalo contínuo $[l, r]$?

Motivação: Target Sum

- Dado um vetor ordenado de tamanho n e um inteiro x , reporte se há 2 elementos cuja soma é igual a x .
- Exemplo: $n = 6$, $x = 11$.

$\{1, 3, 4, 6, 8, 12\}$

Motivação: Target Sum

- Solução ingênua: testar todos os pares possíveis. Solução $O(n^2)$ de tempo e $O(1)$ de memória.

```
1  bool tem = false;
2  for( int i=0; i<n; i++ ){
3      for( int j=i+1; j<n; j++ ){
4          if( v[i]+v[j] == x ){
5              // os elementos i e j são uma resposta possível
6              tem = true;
7          }
8      }
9  }
```

Motivação: Target Sum

- Solução ótima: utilizando dois ponteiros. Solução $O(n)$ de tempo e $O(1)$ de memória.

Dois Ponteiros

- A ideia por trás dessa técnica é iterar dois ponteiros ao longo de um vetor e, com eles, manter salva a resposta, que pode ser um valor ou um intervalo de valores dentro do array.
- Cada ponteiro itera apenas uma vez pelo array, mantendo a complexidade $O(n)$.
- Às vezes, ambos são inicializados juntos no começo do vetor, às vezes um no final e outro no início ou ambos no final. Isso varia de acordo com o problema sendo resolvido.

Motivação: Target Sum

- Repare que se escolhermos 2 elementos quaisquer no vetor para serem nossa resposta, apenas 3 possibilidades podem ocorrer:
 - ▶ A soma é igual a x .
 - ★ Nesse caso encontramos a resposta.
 - ▶ A soma é menor que x .
 - ★ Nesse caso queremos trocar um dos elementos por um valor maior.
 - ▶ A soma é maior que x .
 - ★ Nesse caso queremos trocar um dos elementos por um valor menor.

Motivação: Target Sum

- Podemos nos aproveitar do fato de que o array é ordenado para escolher a forma como vamos percorrer os elementos: ao colocarmos um ponteiro em cada ponta do array, garantimos que ao decrementar o ponteiro da direita, r , estamos reduzindo a soma e, ao incrementar o ponteiro da esquerda, estaremos aumentando a soma final. Sendo assim, nosso algoritmo ficaria:
 - ▶ A soma é igual a x .
 - ★ Nesse caso retornamos a resposta.
 - ▶ A soma é menor que x .
 - ★ Nesse caso incrementamos o ponteiro da esquerda.
 - ▶ A soma é maior que x .
 - ★ Nesse caso decrementamos o ponteiro da direita.

Motivação: Target Sum

$\{\underline{1}, 3, 4, 6, 8, \underline{12}\}$

Motivação: Target Sum

$\{\underline{1}, 3, 4, 6, \underline{8}, 12\}$

Motivação: Target Sum

$$\{1, \underline{3}, 4, 6, \underline{8}, 12\}$$

Motivação: Target Sum

- Solução ótima: utilizando dois ponteiros. Solução $O(n)$ de tempo e $O(1)$ de memória.

```
1
2  bool tem = false;
3  int l=0, r=n-1;
4
5  while( l<r ){
6      if( v[l]+v[r] == x ){
7          // os elementos l e r são uma resposta possível
8          tem = true;
9          break;
10     }else if( v[l]+v[r] < x ){
11         l++;
12     }else if( v[l]+v[r] > x ){
13         r--;
14     }
15 }
```

Por que não perdemos resposta?

Imagine que $v[l] + v[r] < x$.

Como o vetor é ordenado, trocar r por qualquer posição menor só vai deixar o segundo número menor ou igual.

$$v[l] + v[r'] \leq v[l] + v[r] < x$$

Então $v[l]$ não fecha par com ninguém até r . Podemos descartar l .

► **Pergunta:** Agora faça o raciocínio contrário: se $v[l] + v[r] > x$, por que podemos descartar r ?

Problema 1: Books

- Nós temos n livros enfileirados na nossa prateleira e uma quantidade t de tempo livre. Cada livro i gasta $a[i]$ minutos para ser lido. Queremos achar qual a maior quantidade de livros que conseguimos ler dentro de t minutos, lendo os livros em sequência na nossa prateleira, sendo que podemos escolher começar a partir de qualquer livro.
- Exemplo: $n = 4$, $t = 5$.

$$a = \{3, 1, 2, 1\}$$

Problema 1: Books

- Nós temos n livros enfileirados na nossa prateleira e uma quantidade t de tempo livre. Cada livro i gasta $a[i]$ minutos para ser lido. Queremos achar qual a maior quantidade de livros que conseguimos ler dentro de t minutos, lendo os livros em sequência na nossa prateleira, sendo que podemos escolher começar a partir de qualquer livro.
- Exemplo: $n = 4$, $t = 5$.

$$a = \{3, 1, 2, 1\}$$

- resposta = 3, os livros $\{3, \underline{1}, \underline{2}, \underline{1}\}$

Problema 1: Books

- Solução $O(n)$ de tempo:
- Utilizando dois ponteiros, l e r , ambos inicializados no começo do array, incremente o ponteiro da esquerda toda vez que a soma de todos os elementos no intervalo de l a r for maior que t e o da direita toda vez que for menor ou igual.
- Toda vez que o intervalo atual de livros gastar um tempo menor ou igual a t , verifique se ele é maior que a resposta que temos até agora e salve o valor.

Problema 1: Books

```
1  int livros = 0;
2  int l = 0, r = 0;
3  int intervalo = a[0];
4  while(r<n){
5      if( intervalo >t ){
6          intervalo -= a[l];
7          l++;
8      } else {
9          livros = max(livros, r-l+1);
10         r++;
11         if(r<n){
12             intervalo += a[r];
13         }
14     }
15 }
```

Agora: Sliding Window

Sliding Window é um caso de two pointers em que os dois ponteiros andam na mesma direção.

A janela atual é o intervalo:

$$\begin{array}{c} [l, r] \\ [3, 1, 2, 1] \\ \quad \quad \quad l \quad \quad r \end{array}$$

- **Pergunta:** O intervalo $[l, r]$ representa elementos contínuos ou elementos escolhidos em qualquer posição?

Ideia da janela

Mantemos:

- l : início da janela;
- r : fim da janela;
- $soma$: soma dos tempos dentro da janela;
- ans : maior tamanho válido encontrado.

Regra:

- adicionamos $a[r]$ na janela;
- enquanto $soma > t$, removemos $a[l]$ e fazemos $l++$;
- se $soma \leq t$, atualizamos a resposta.

► **Pergunta:** Quando a janela fica pesada demais, por qual lado tiramos livros?

Simulação: Books

$$a = [3, 1, 2, 1], \quad t = 5$$

passo	adiciona/remove	janela	soma	ans
1	adiciona 3	[3]	3	1
2	adiciona 1	[3, 1]	4	2
3	adiciona 2	[3, 1, 2]	6	inválida
4	remove 3	[1, 2]	3	2
5	adiciona 1	[1, 2, 1]	4	3

Código: Books com Sliding Window

Suponha que a , n e t já foram lidos.

```
1  int l = 0;
2  long long soma = 0;
3  int ans = 0;
4
5  for(int r = 0; r < n; r++){
6      soma += a[r];
7
8      while(soma > t){
9          soma -= a[l];
10         l++;
11     }
12
13     ans = max(ans, r - l + 1);
14 }
15 cout << ans << endl;
```

Modelo mental de Sliding Window

Pense na janela como uma mochila:

- o ponteiro r coloca um item novo;
- se a mochila passa do limite, l tira itens antigos;
- em cada momento válido, tentamos melhorar a resposta.

Quando Sliding Window funciona bem?

Funciona muito bem quando:

- estamos procurando um **subarray/substring contínuo**;
 - os ponteiros só precisam andar para frente;
 - quando a janela fica inválida, remover da esquerda ajuda;
 - a condição é monotônica.
- ★ **Dica:** Para soma com limite, a versão simples costuma exigir números não negativos. Se houver negativos, a soma pode diminuir ao adicionar um elemento, e a regra pode quebrar.
- **Pergunta:** Por que números negativos podem atrapalhar uma janela baseada em soma?

Template geral de Sliding Window

```
1  int l = 0;
2  long long estado = 0;
3  int ans = 0;
4
5  for(int r = 0; r < n; r++){
6      estado += a[r];
7
8      while(janela_invalida){
9          estado -= a[l];
10         l++;
11     }
12
13     ans = max(ans, r - l + 1);
14 }
```

- **Pergunta:** O que muda de um problema para outro: o movimento dos ponteiros ou a forma de manter o estado?

Problema 2: Counting Kangaroos is Fun

- Há n cangurus com “bolsas”(marsúpios). Cada canguru possui um tamanho (número inteiro). Um canguru a pode ser carregado por um canguru b se e somente se o tamanho de b é pelo menos 2 vezes maior que o de a . Cada canguru pode carregar um único canguru e um canguru que é carregado não pode carregar outro canguru.
- Sempre que um canguru é carregado, ele não é mais visível. Reporte o menor número de cangurus visíveis em um plano de carregamento de cangurus.
- Exemplo: $\{2, 2, 4, 5, 6, 7, 8, 9\}$

Problema 2: Counting Kangaroos is Fun

- Há n cangurus com “bolsas”(marsúpios). Cada canguru possui um tamanho (número inteiro). Um canguru a pode ser carregado por um canguru b se e somente se o tamanho de b é pelo menos 2 vezes maior que o de a . Cada canguru pode carregar um único canguru e um canguru que é carregado não pode carregar outro canguru.
- Sempre que um canguru é carregado, ele não é mais visível. Reporte o menor número de cangurus visíveis em um plano de carregamento de cangurus.
- Exemplo: $\{2, 2, 4, 5, 6, 7, 8, 9\}$
- resposta = 5, pois os 3 primeiros cangurus podem ser carregados pelos 3 últimos.

Problema 2: Counting Kangaroos is Fun

- Solução $O(n)$ de tempo:
- Como o número máximo de relações entre cangurus é $n/2$, podemos particionar os cangurus entre os $n/2$ menores, os que podem ser carregados, e $n/2$ maiores os que podem carregar os menores. Cada ponteiro será inicializado em uma das duas metades, l e r .
- Verifique se o menor canguru, na posição l , pode ser carregado pelo menor canguru na segunda metade do array, na posição r .
- Se for possível carregar, então devemos subtrair 1 da resposta, pois o canguru não está mais visível. Senão, devemos incrementar r até encontrar um canguru maior para carregar o menor de todos, l .

Solução problema 2: Counting Kangaroos is Fun

```
1  sort(v.begin(), v.end());
2  int l=0, r=n/2;
3  int tira=0;
4
5  for( ; l<n/2 and r<n; ){
6      if( v[l]*2 <= v[r] ){
7          l++, r++;
8          tira++;
9      }else{
10         r++;
11     }
12 }
13 cout << n-tira << endl;
```

Erros comuns

- Usar `while(l <= r)` quando não pode usar o mesmo elemento duas vezes.
- Esquecer de ordenar quando o padrão depende de ordem.
- Ordenar e perder os índices originais.
- Atualizar `ans` com uma janela inválida.
- Esquecer de remover `a[1]` antes de fazer `l++`.
- Usar sliding window de soma com números negativos sem verificar se a lógica ainda funciona.

Como reconhecer na hora da prova?

Leia o enunciado e procure sinais:

Sinal no problema	Técnica provável
par com soma alvo em vetor ordenado	pontas opostas
maior intervalo com soma $\leq k$	sliding window
substring sem repetir	sliding window com frequência
3 valores somando alvo	fixa um índice + two pointers
parear pequenos com grandes	ordenar + two pointers

► **Pergunta:** Se o problema fala “subarray”, qual técnica deve acender uma luz na cabeça?

Estratégia de resolução

Quando achar que é Two Pointers:

1. Faça uma solução ingênua primeiro mentalmente.
2. Veja o que está sendo repetido à toa.
3. Pergunte: os ponteiros podem andar só para frente?
4. Defina claramente o que cada ponteiro significa.
5. Defina quando mover cada ponteiro.
6. Teste em um exemplo pequeno na mão.

Resumo final

- Two pointers usa índices para reduzir busca repetida.
- Pontas opostas funciona muito bem com vetor ordenado.
- Sliding window mantém um intervalo contínuo $[l, r]$.
- O segredo é saber **quando mover cada ponteiro**.
- A complexidade costuma ser $O(n)$, porque cada ponteiro anda no máximo n vezes.

Lista de treino sugerida

Comece pelos mais básicos:

1. Magical Array
2. Merging Arrays
3. Number of Smaller
4. Number of Equal
5. HOTELS
6. Ternary String
7. Small Sum
8. Big Sum
9. Small Set
10. Seat Arrangements
11. Rooms
12. Kangaroos
13. Spread
14. Third-Party
15. Glider

Perguntas?

Próximo passo: resolver problemas pequenos até o padrão ficar automático.